Theory and Methods for Track Cycling Simulation and Aerodynamic Drag (CdA) Fitting

David Domonoske

August 28, 2026

Contents

1 Introduction **3**

2 Forward Simulation Model **3**
2.1 Equation of motion . . . 4
2.2 Rolling resistance . . . 4
2.3 Aerodynamic drag . . . 4
2.4 Propulsive force . . . 5
2.5 Numerical integration . . . 5
2.6 Outputs . . . 6

3 Cornering Model **6**
3.1 Framing: a quasi-static lean, not a dynamical degree of freedom . . . 6
3.2 Lean angle . . . 7
3.3 Center-of-mass offset and wheel-speed geometry . . . 7
3.4 Energy-feedback term . . . 8
3.4.1 Explicit s-dependence of theta . . . 8
3.4.2 Vertical-energy acceleration term . . . 8
3.4.3 Implicit feedback coefficient . . . 9
3.5 Combined equation of motion . . . 9
3.6 Low-speed floor . . . 10

4 Track Geometry Model **10**
4.1 Idealized track shape . . . 10
4.2 Corner radius . . . 10
4.3 Curvature profile and transition smoothing . . . 10
4.4 Position reconstruction . . . 11

5 Aerodynamic Drag (CdA) Fitting **12**
5.1 Acceleration from recorded data . . . 12
5.2 Rearranged force balance . . . 12
5.3 Closed-form least-squares fit . . . 13

5.4 Lap-phase search 13

6 Assumptions and Limitations **13**

A Notation, Units, and Parameters **15**

1 Introduction

This document describes the physics, numerical methods, and modeling assumptions behind the two central calculations in the WOTT cycling performance application:

1. **Forward race simulation**, implemented by `IPCalculator`: given a rider's physical parameters, a target power plan, and environmental and track conditions, integrate the equations of motion forward in time to predict velocity, position, split times, and power output over the course of a race.

2. **Aerodynamic drag fitting**, implemented by `CdAFitter`: given recorded speed and power data from a `.fit` file (the standard recording format written by GPS/ANT+ cycling computers), invert the same equations of motion to estimate a rider's aerodynamic drag area, C_dA, from field data.

Both calculations share a single underlying physical model: a point mass moving along a one-dimensional track coordinate s, subject to rolling resistance, aerodynamic drag, and a rider-supplied propulsive force, together with an optional quasi-static cornering correction that activates whenever track curvature is present, with further geometric and energetic refinements activating when the rider's center-of-mass height is also known. Section 2 develops this equation of motion and its numerical solution. Section 3 derives the cornering correction in detail, since it is the most involved part of the model. Section 4 describes how the track's curvature profile is constructed from a small set of geometric parameters. Section 5 shows how the forward model is inverted to estimate C_dA from measured ride data. Section 6 collects the simplifying assumptions made throughout and states their limitations plainly. Section A tabulates every symbol used in this document, its SI unit, its default value where one exists, and the corresponding function or variable name in the implementation (`calcs.py`).

State and notation conventions

The forward simulation's state at time t is the pair

$$y(t) = \big(v(t),\ s(t)\big), \tag{1}$$

where v is the rider's speed along the direction of travel, in m/s, and s is the distance traveled along the track, in m. All quantities in this document are expressed in SI units unless stated otherwise. The one exception is velocity as reported to the user, which is converted to km/h purely for display; this conversion is never used internally and is noted again where it occurs (Section 2.6).

2 Forward Simulation Model

The forward simulation is implemented by `IPCalculator` and solved numerically with `scipy.integrate.so` the equation of motion follows.

2.1 Equation of motion

The model treats the rider and bicycle as a single point mass m moving along the track's arc-length coordinate s. On a flat, straight section of track, Newton's second law along the direction of travel gives

$$m\,\frac{dv}{dt} = F_{rr} + F_{ad} + F_p(1-\eta), \tag{2}$$

where F_{rr} is the rolling resistance force, F_{ad} is the aerodynamic drag force, F_p is the propulsive force delivered before drivetrain losses, and η (`mech_losses`) is the fractional mechanical loss through the drivetrain. Dividing by m gives the straight-line acceleration

$$a_{\text{power}} = \frac{F_{rr} + F_{ad} + F_p(1-\eta)}{m}. \tag{3}$$

When the track has curvature, an additional cornering correction is added to Eq. (3); that correction is developed in full in Section 3, since it depends on quantities not yet introduced here.

2.2 Rolling resistance

On level, upright riding, rolling resistance is the familiar $F_{rr,0} = -mgC_{rr}$, opposing the direction of travel, where g is the acceleration of gravity and C_{rr} is the rolling resistance coefficient (`envir.crr`). When the rider is leaned over at angle θ through a corner (Section 3 derives θ), the model treats the tire-track contact *as if* the rider were riding a surface banked at angle θ, rather than balancing via tire friction on a flat track: no lateral friction is needed to supply the centripetal force under this idealization, but the normal force increases, since the same weight mg must be supported through a smaller effective vertical component:

$$N(\theta) = \frac{mg}{\cos\theta}. \tag{4}$$

Because rolling resistance is proportional to the normal force, this gives

$$F_{rr}(\theta) = -\frac{mgC_{rr}}{\cos\theta}. \tag{5}$$

At $\theta = 0$ (straights, or when no cornering model is configured), $\cos\theta = 1$ and Eq. (5) reduces to the standard flat result $F_{rr,0}$.

2.3 Aerodynamic drag

The aerodynamic drag force follows the standard quadratic drag equation,

$$F_{ad} = -\tfrac{1}{2}\rho\, C_d A\, v^2, \tag{6}$$

where ρ is air density (`envir.air_density`) and C_dA is the rider's aerodynamic drag area: the product of drag coefficient and frontal area, treated here as a single lumped parameter with units of m^2. C_dA is exactly the quantity that Section 5 estimates from recorded ride data by inverting this same force balance.

2.4 Propulsive force

The rider's target output is specified as a power plan: a piecewise sequence of (start time, power, duration) segments giving the commanded power $P(t)$ at any instant. Converting a target power into a propulsive force requires dividing by speed, $F = P/v$, which is singular as $v \to 0$ and is physically unrealistic at low speed in any case, since no rider can deliver unbounded force. The model therefore caps the propulsive force at a maximum traction force $F_{\max}$ (`max_force`, default 200 N):

$$F_p(v,t) = \begin{cases} F_{\max}, & v < P(t)/F_{\max} \quad \text{(torque-limited)} \\ P(t)/v, & v \geq P(t)/F_{\max} \quad \text{(power-limited)}, \end{cases} \tag{7}$$

with $F_p = 0$ whenever $v = 0$ and $P(t) \leq 0$. This is the standard torque-limited/power-limited force-velocity curve used in vehicle propulsion models.

2.5 Numerical integration

Equation (2) (extended with the cornering correction of Section 3) is a coupled, nonstiff, first-order ODE system in the state $y = (v, s)$ of Eq. (1). It is solved with `scipy.integrate.solve_ivp` using its default explicit Runge-Kutta method (RK45, the Dormand-Prince pair) and default error tolerances. RK45 is adaptive: at every step it estimates the local truncation error and grows or shrinks the next step to keep that error within tolerance. The only control the simulation imposes on this process is `max_step`, an upper bound (discussed below); below that bound the solver's own error control still governs the step actually taken, and it does shrink on its own when the dynamics call for it – for instance, right at a power-plan segment boundary, where the commanded power $P(t)$ jumps discontinuously between segments (Section 2.4), or near the torque-limited/power-limited crossover speed $P(t)/F_{\max}$ in Eq. (7), where the force curve has a kink.

The solver is given a terminal event that stops integration exactly when the rider crosses the target race distance,

$$e(t,y) = s(t) - s_{\text{race}} = 0, \tag{8}$$

detected as a rising zero-crossing (`direction=1`) so integration halts on the sample where the rider first reaches s_{race} (`race_distance`, default 4000 m). SciPy locates this crossing by root-finding on its dense-output polynomial, giving sub-step accuracy in the reported finish time and position; the run raises an error if the event never fires within the solve horizon $t_{\max}$ (default 300 s).

Results are additionally requested on a fixed, uniformly spaced grid (`t_eval`) of $n = \lceil t_{\max}/dt \rceil$ points spanning $[0, t_{\max})$ – a spacing of $t_{\max}/n$, equal to dt when $t_{\max}$ is an exact multiple of it and slightly under dt otherwise – independent of the solver's own internal adaptive step choices. When track geometry is configured, the solver's `max_step` is capped at dt; when no track geometry is present it is left unbounded (`numpy.inf`) so RK45 is free to take its own, larger, error- controlled steps. The reason for this cap is the curvature profile itself: Section 4 shows that $\kappa(s)$ is smoothed into a ramp between zero and its cornering value over a short transition zone (default 30 m), which a fast-moving rider can cross in well under a second. RK45's own error control, tuned to the comparatively smooth power and

drag dynamics, may grow its step size to several seconds without raising any error, silently stepping over a transition zone between two `t_eval` samples and producing a visibly jagged velocity trace through the corner, since the solver's dense-output polynomial has no way to know a sharp feature lies between its chosen steps. Capping `max_step` at dt forces the solver to evaluate the right-hand side within every transition zone, at some cost to integration speed.

2.6 Outputs

The simulation produces time series of velocity $v(t)$, position $s(t)$, and power $P(t)$. Velocity is stored internally in m/s and converted to km/h (a factor of 3.6) only when reported to the user. Reported power at each sample re-applies the same force cap as Eq. (7), $P_{\text{report}} = \min(F_{\max} v,\ P(t))$, vectorized over the full solution.

Lap splits are computed at evenly spaced distance markers (default every 125 m up to 4000 m) by linear interpolation between the two position samples straddling each split distance, giving split times at better than one `t_eval` step of resolution. For CSV export, the velocity and power traces are resampled onto a uniform 1 s grid using time-weighted averaging via cumulative trapezoidal integration, rather than nearest-sample decimation, so that the resampled curve's area (and therefore total energy and total distance) matches the full-resolution simulation.

3 Cornering Model

Whenever the track's curvature profile $\kappa(s)$ (Section 4) is nonzero, the simulation adds a lean-angle and banked-rolling-resistance correction to the equation of motion (Sections 2.2 and 3.2). If the rider's center-of-mass height h (`com_height_m`) is also known, two further corrections activate: the wheel-speed geometric correction and the energy-feedback term (Sections 3.3 and 3.4). This section derives each term in turn.

3.1 Framing: a quasi-static lean, not a dynamical degree of freedom

The lean angle θ derived below is easy to over-read as a full rigid-body treatment; it is not one. θ is *algebraically slaved* to the instantaneous speed v and track curvature κ at every instant; it is not an independent state variable with its own rotational inertia, and the model does not track a separate roll rate. There is also no friction-circle or traction-limit check: θ is computed unconditionally from v and κ regardless of whether the tire could actually supply the implied lateral friction. This is a deliberate simplification, revisited in Section 6, appropriate for the smooth, moderate-curvature track geometry of Section 4 but not a general-purpose vehicle dynamics model.

3.2 Lean angle

For a bicycle in a steady, flat turn of radius $r = 1/\kappa$ at speed v, the classic no-slip lean-angle balance equates the lateral force required for circular motion, mv^2/r, with the horizontal component of gravity acting through the lean:

$$\tan\theta = \frac{v^2}{gr} = \frac{v^2\kappa}{g}. \tag{9}$$

Solving for θ,

$$\theta(v,\kappa) = \arctan\left(\frac{v^2\kappa}{g}\right). \tag{10}$$

On a straight ($\kappa = 0$) or at rest ($v = 0$), $\theta = 0$ and every term introduced in this section vanishes, so the model reduces exactly to Section 2 with no cornering configured.

3.3 Center-of-mass offset and wheel-speed geometry

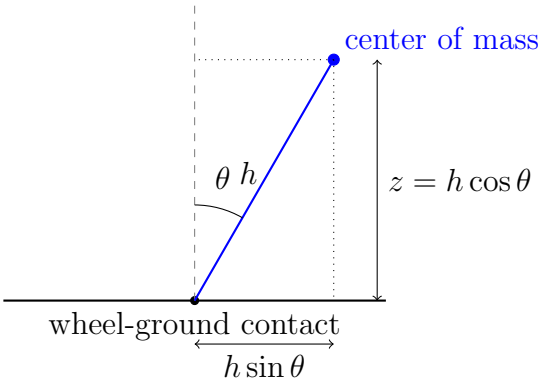

Figure 1: Lean geometry viewed from behind the rider: the center of mass sits a distance h from the wheel-ground contact point at lean angle θ from vertical, with horizontal offset $h\sin\theta$ and height $z = h\cos\theta$ above the contact point.

The state variable v represents the speed of the rider's center of mass (Fig. 1). When leaned at angle θ, the center of mass sits a horizontal distance $h\sin\theta$ inboard of the wheel-ground contact line, tracing a smaller effective turn radius than the wheels themselves, which remain on the track surface: writing $r = 1/\kappa$ for the wheels' turn radius, the center of mass turns on a radius $r - h\sin\theta$. For the same angular sweep of the turn, both the wheels and the center of mass sweep the same angle in the same time, so their speeds scale directly with radius, $v/v_{\text{wheel}} = (r - h\sin\theta)/r = 1 - \kappa h\sin\theta$. Solving for the wheel speed,

$$v_{\text{wheel}}(v,\kappa,\theta) = \frac{v}{1 - \kappa h\sin\theta}, \tag{11}$$

with the denominator floored at 0.01 in the implementation as a numerical safeguard against extreme combinations of κ, h, and θ driving it toward zero. This matters because track curvature $\kappa(s)$ is indexed by arc length along the track surface – the wheels' path – not

by the center of mass's path. Consequently the position state advances at the wheel speed rather than the center-of-mass speed,

$$\frac{ds}{dt} = v_{\text{wheel}}, \tag{12}$$

Equation (12) itself only updates the position state; it is not a term in dv/dt. The wheel speed v_{wheel} it defines, however, is reused below in Section 3.4's energy-feedback term, so the cornering correction as a whole does still affect dv/dt, just not through this equation.

Because the race-distance terminal event of Section 2.5 triggers when s – the same coordinate advanced via Eq. (12) – reaches s_{race}, the tracked race distance is a track-surface (wheel-path) distance, matching how a race distance is physically marked on a track. A direct consequence is that the rider's center of mass itself covers a slightly shorter cumulative distance than s_{race} over the course of a full run, since it cuts inside the wheels' path through every corner.

3.4 Energy-feedback term

As the rider leans into and out of a corner, the center of mass also moves *vertically*: writing $z \equiv h\cos\theta$ for its height above a fixed reference, z decreases as θ grows from zero – leaning over lowers the center of mass, the same way tipping a rigid rod from vertical lowers its tip. By conservation of mechanical energy, this loss of height converts into a small *gain* in forward speed on the way into a corner; the reverse conversion, speed converting back into height, gives a small *loss* of speed coming back upright on the way out. This subsection derives that acceleration contribution, denoted a_{energy}.

3.4.1 Explicit s-dependence of theta

Differentiating Eq. (9) implicitly with respect to arc length s at fixed v, using $\frac{d}{ds}\tan\theta = \sec^2\theta\,\partial\theta/\partial s$ and the identity $\sec^2\theta = 1 + \tan^2\theta$, gives

$$\sec^2\theta\,\left.\frac{\partial\theta}{\partial s}\right|_v = \frac{v^2}{g}\frac{d\kappa}{ds} \quad\Longrightarrow\quad \left.\frac{\partial\theta}{\partial s}\right|_v = \cos^2\theta\,\frac{v^2}{g}\frac{d\kappa}{ds}, \tag{13}$$

the rate of change of lean angle due to the changing curvature alone. This derivative is well-defined everywhere: Section 4.3 constructs $\kappa(s)$ as a smoothed, differentiable profile rather than a step function, so $d\kappa/ds$ is a genuine, bounded function of s rather than a delta spike at each corner boundary. (The implementation computes this equivalent form as $1/(1+x^2)$ with $x \equiv \tan\theta = v^2\kappa/g$, identical to $\cos^2\theta$ by the same identity.)

3.4.2 Vertical-energy acceleration term

The center-of-mass height above a fixed reference is, up to a constant, $h\cos\theta$, so its vertical velocity is $\frac{dz}{dt} = -h\sin\theta\,\frac{d\theta}{dt}$. Energy conservation on the combined kinetic and potential terms, $\frac{d}{dt}\left(\frac{1}{2}v^2 + gz\right) = 0$ for the lean-driven exchange alone, gives $v\left.\frac{dv}{dt}\right|_{\text{lean}} = gh\sin\theta\,\frac{d\theta}{dt}$, or

$$a_{\text{energy}} = \frac{gh\sin\theta}{v}\frac{d\theta}{dt}. \tag{14}$$

Only the arc-length-driven part of $d\theta/dt$ – that is, $\partial\theta/\partial s|_v \cdot ds/dt$, using the wheel speed of Eq. (12) – is applied explicitly here:

$$a_{\text{energy}}^{(s)} = \frac{gh\sin\theta}{v}\left(\frac{\partial\theta}{\partial s}\right) v_{\text{wheel}}. \tag{15}$$

The remaining part of $d\theta/dt$, driven by v itself changing, is handled separately in the next subsection: since θ depends on v (Eq. (10)), and v is precisely the quantity being solved for, folding that dependence directly into Eq. (14) would make the right-hand side of the ODE depend on dv/dt itself.

3.4.3 Implicit feedback coefficient

Differentiating Eq. (9) implicitly with respect to v at fixed κ the same way gives the v-driven part of $d\theta/dt$:

$$\frac{\partial\theta}{\partial v} = \cos^2\theta\,\frac{2v\kappa}{g}. \tag{16}$$

Substituting the full chain rule $d\theta/dt = \partial\theta/\partial s|_v\, v_{\text{wheel}} + \partial\theta/\partial v \cdot dv/dt$ into Eq. (14), and adding this lean-driven acceleration to the non-cornering acceleration of Eq. (3), gives the full equation of motion in the form

$$\frac{dv}{dt} = a_{\text{power}} + a_{\text{energy}}^{(s)} + C(v,\kappa,\theta,h)\cdot\frac{dv}{dt}, \tag{17}$$

self-referential in dv/dt through θ's own dependence on v, where

$$C(v,\kappa,\theta,h) = 2h\kappa\sin\theta\cos^2\theta. \tag{18}$$

Here every explicit factor of v and g introduced by the $gh\sin\theta/v$ prefactor of Eq. (14) and the $2v\kappa/g$ factor of Eq. (16) cancels, leaving C independent of both.

3.5 Combined equation of motion

Because C enters Eq. (17) linearly, dv/dt can be solved for explicitly rather than treated as a second implicit unknown requiring its own root-find at every solver step. Solving Eq. (17) for dv/dt gives the full cornering-corrected equation of motion:

$$\frac{dv}{dt} = \frac{a_{\text{power}} + a_{\text{energy}}^{(s)}}{\max\big(1-C,\ 0.1\big)}. \tag{19}$$

The floor on the denominator is a numerical safeguard against C approaching 1, which would only occur at combinations of lean, curvature, and speed far outside any realistic riding condition; it has no physical interpretation. Together with Eq. (12), Eq. (19) completes the right-hand side of the ODE integrated in Section 2.5.

As a consistency check, this construction is antisymmetric between corner entry and corner exit: because $d\kappa/ds$ changes sign between the entry and exit transition ramps of Section 4 while v and θ are approximately unchanged, $a_{\text{energy}}^{(s)}$ takes equal and opposite values entering and exiting a corner at the same speed. Over a full lean-in and lean-out cycle at constant v, the term therefore integrates to zero net effect on speed, consistent with energy conservation.

3.6 Low-speed floor

The energy-feedback terms of this section – a_{energy} and the feedback coefficient C – are set to zero whenever $v < V_{\text{eps}}$ (0.1 m/s). This avoids division-by-near-zero in Eq. (14), the only one of the two with an explicit $1/v$ term; Eq. (18) has no such term (it depends only on κ, h, and θ) but is zeroed alongside it for consistency, since both represent the same near-zero-speed cornering-energy exchange. (Lean angle θ and the banked rolling resistance of Section 2.2 are not gated this way; both remain well-defined at $v = 0$, where $\theta \to 0$ smoothly.) This is a numerical safeguard for the simulation's start condition, not a distinct physical regime.

4 Track Geometry Model

The cornering model of Section 3 requires a curvature profile $\kappa(s)$ describing how sharply the track turns at each point along its length. This profile is generated by `TrackShape` from a small number of geometric parameters and is also used purely for visualizing the track shape in the application's user interface.

4.1 Idealized track shape

The track is modeled as an idealized oval: two straight sections and two semicircular corners, in the style of a running track or velodrome. Given total track length L (`track_length`) and a dimensionless fraction $c \in [0.20, 1.0]$ (`corners`) specifying what portion of L is devoted to cornering, the arc length of each straight and each corner is

$$\ell_{\text{straight}} = \frac{L(1-c)}{2}, \qquad \ell_{\text{corner}} = \frac{Lc}{2}. \tag{20}$$

The user-facing control for c is labeled from "hotdog" at its minimum ($c = 0.20$, long straights and tight, brief corners) to "circle" at its maximum ($c = 1.0$, no straights at all, i.e. a single circular track).

4.2 Corner radius

Each corner is modeled as an exact semicircular arc: a 180° turn subtending arc length ℓ_{corner}. Since arc length equals radius times swept angle, and the swept angle of a semicircle is π, the (fixed) corner radius is

$$r = \frac{\ell_{\text{corner}}}{\pi}, \tag{21}$$

coinciding with the instantaneous turn radius $1/\kappa$ of Section 3.2 only on the corner plateau, not through the smoothed transition ramps of Section 4.3. The corner curvature is then simply $\kappa_{\text{corner}} = 1/r$.

4.3 Curvature profile and transition smoothing

Without smoothing, curvature along the track would be a step function: zero on the straights, $1/r$ on the corner arcs, with a discontinuous jump at each straight-to-corner boundary.

Physically, no rider leans instantaneously; the transition into and out of a corner takes some finite distance. `TrackShape` approximates this by passing the raw step-function curvature through a box (moving-average) filter of width `transition` (default 30 m), producing a ramp-plateau-ramp curvature profile at each corner entry and exit rather than a sharp step. This is not a true clothoid (an Euler spiral, the curvature-linear-in-arc-length transition used in road and rail design) but serves an analogous smoothing purpose with substantially less implementation complexity, at the cost of a curvature profile whose transition shape is set by the filter kernel rather than by an exact geometric construction. It is this smoothed transition ramp that motivates the `max_step` restriction discussed in Section 2.5.

This smoothing also leaves the track's total accumulated turning unchanged, *provided* each straight is long enough that the box filter's averaging window – half-width transition/2 on either side – does not reach past the straight into the raw curvature of an adjacent corner. A box filter is a normalized moving average, so it redistributes curvature locally without changing its integral as long as the signal it averages is unchanged wherever the window reaches past the domain's ends; that holds here whenever $\ell_{\text{straight}}/2 \geq \text{transition}/2$, i.e. $L(1-c)/4 \geq \text{transition}/2$, which the application's default parameters satisfy ($\ell_{\text{straight}}/2 = 25\,\text{m}$ against a 15 m half-window at the default 250 m track and $c = 0.6$) but fails as $c \to 1$, where the straights vanish entirely. Within that regime,

$$\int_0^L \kappa(s)\, ds = 2\pi \tag{22}$$

exactly, independent of the transition length: each of the two corners is an exact semicircle contributing $\kappa_{\text{corner}}\,\ell_{\text{corner}} = \pi$ radians of turning by Eq. (21), and smoothing only relocates *where* that turning happens along s (Section 4.4 verifies the resulting loop closure numerically).

4.4 Position reconstruction

For visualization only – this reconstruction has no effect on the physics of Sections 2 and 3 – the track's heading and Cartesian centerline coordinates are recovered from the curvature profile by the standard Frenet-style construction: heading is the running integral of curvature, and position is the running integral of the heading's cosine and sine components,

$$\phi(s) = \int_0^s \kappa(s')\, ds', \tag{23}$$

$$X(s) = \int_0^s \cos\phi(s')\, ds', \tag{24}$$

$$Y(s) = \int_0^s \sin\phi(s')\, ds'. \tag{25}$$

Each integral is evaluated numerically by cumulative trapezoidal integration. As a consequence of Eq. (22), the reconstructed centerline closes on itself to within 1 m of the starting point after one full lap, for the straight/transition combinations described in Section 4.3.

5 Aerodynamic Drag (CdA) Fitting

`CdAFitter` inverts the equation of motion of Sections 2 and 3: instead of integrating forward from a known C_dA to predict $v(t)$, it takes a measured (t, v, P) trace from a rider's `.fit` file and solves for the one remaining unknown, C_dA. Given a selected window of recorded samples, the rider's acceleration can be computed directly by differentiating the recorded speed trace, leaving Eq. (2) (extended with the cornering forces of Section 3) as a single linear equation in C_dA, since every other quantity in the force balance – mass, rolling resistance, propulsive force, and the cornering corrections – can be computed directly from the recorded data and the rider's/environment's known parameters.

5.1 Acceleration from recorded data

Recorded samples are filtered to those with $v > V_{\text{eps}}$, and the acceleration dv/dt is estimated by a central finite difference (`numpy.gradient`) on the raw, non-uniformly spaced recording timestamps, with no smoothing applied to the speed trace beforehand. Distance traveled within the selected window is obtained by cumulative trapezoidal integration of the recorded speed, giving a cumulative distance dist_i at each sample i from the start of the window – using recorded (center-of-mass) speed rather than the wheel speed v_{wheel} of Eq. (11), a small approximation revisited in Section 6 – rather than from a separate odometer or GPS-distance field. Together with the phase offset s_0 of Section 5.4, this gives each sample's track position, $s_i = s_0 + \text{dist}_i$, used for the curvature lookups $\kappa(s_i)$ that feed the cornering corrections below.

5.2 Rearranged force balance

Unlike the forward simulation, dv/dt here is already known – estimated directly from the recorded speed trace in Section 5.1 – so the circularity that motivated splitting a_{energy} into an explicit s-driven part (Eq. (15)) and an implicit v-driven feedback coefficient C (Eq. (18)) does not arise here: the fit evaluates the full chain rule directly, $d\theta/dt = \partial\theta/\partial s|_v \, v + \partial\theta/\partial v \cdot dv/dt$ (Eqs. (13) and (16), using recorded speed v in place of v_{wheel}, Section 5.1), and substitutes it into the general Eq. (14) to get a_{energy} directly.

Rearranging Eq. (2) (with F_{ad} from Eq. (6) and the cornering-corrected rolling resistance and energy terms of Section 3) to isolate the drag force gives

$$F_{ad} = m\,\frac{dv}{dt} - F_p(1-\eta) - F_{rr}(\theta) - m\,a_{\text{energy}}, \tag{26}$$

where the propulsive force here uses the recorded power directly, $F_p = P(t)/v$, with no traction cap applied (unlike the forward model's Eq. (7)), since $P(t)$ is measured rather than a target to be achieved. Equation (6) gives $F_{ad} = -C_dA \cdot \left(\frac{1}{2}\rho v^2\right)$, always negative since drag opposes motion; working instead with the drag *magnitude* $D \equiv -F_{ad} = C_dA \cdot \left(\frac{1}{2}\rho v^2\right)$, a positive quantity, and negating Eq. (26) accordingly, gives

$$x \equiv \tfrac{1}{2}\rho v^2, \qquad y \equiv D = F_p(1-\eta) + F_{rr}(\theta) + m\,a_{\text{energy}} - m\,\frac{dv}{dt}, \tag{27}$$

reducing the fit to a single linear regression, $y = C_dA \cdot x$, evaluated across every sample in the selected window.

5.3 Closed-form least-squares fit

Because Eq. (6) requires $F_{ad} = 0$ at $v = 0$, the correct regression has no intercept term: the fit is constrained to pass through the origin, not merely close to it. The ordinary-least-squares solution to $y = C_dA \cdot x$ with a fixed zero intercept has the closed form

$$C_dA = \frac{\sum_i x_i y_i}{\sum_i x_i^2}, \tag{28}$$

computed directly with no iterative solver:

```
cda = np.sum(x * y) / np.sum(x ** 2)
residual = np.sum((y - x * cda) ** 2)
```

The fit is rejected if $C_dA \leq 0$ (a non-physical result) or if $\sum_i x_i^2 = 0$ – a defensive guard against a zero denominator that, given the $v > V_{\text{eps}}$ filtering of Section 5.1, is not expected to trigger in practice. Independently, the selected window must contain at least two samples with $v > V_{\text{eps}}$ after filtering; fewer raises an error rather than returning a degenerate fit.

5.4 Lap-phase search

The track's curvature $\kappa(s)$ is periodic in position s with fundamental period $L/2$ – one straight-and-corner unit, from Eq. (20) – since the two straight-and-corner units making up a full lap of length L are geometrically identical (Section 4). The phase offset s_0 at which the recorded window began – where on the track, modulo that period, the rider was when recording started – is not directly known from the `.fit` data. Because the residual-versus-s_0 landscape is periodic and, in general, multimodal (a local optimizer seeded arbitrarily could converge to the wrong minimum), `CdAFitter` searches for s_0 in two stages:

1. **Coarse grid search**: the fit of Eq. (28) is evaluated at 150 candidate offsets spaced evenly over the full period $[0, L/2)$, covering every distinct phase the window could have started at.
2. **Refinement**: the best coarse candidate is refined with `scipy.optimize.minimize_scalar` using the bounded Brent method, searching within one grid spacing on either side of the coarse minimum.

At each candidate s_0, the scoring function is the sum of squared residuals of the fit in Eq. (28), $\sum_i \big(y_i - x_i C_dA(s_0)\big)^2$, so s_0 and C_dA are fit jointly: C_dA is solved in closed form for each candidate s_0, and s_0 itself is chosen to minimize the resulting residual. If no track geometry is configured, s_0 is not searched and the fit is evaluated directly with $\kappa \equiv 0$ throughout, in which case the cornering terms of Eq. (26) vanish identically.

6 Assumptions and Limitations

The models in Sections 2 to 5 make a number of simplifying assumptions. They are listed here plainly, so that their limitations are clear to anyone interpreting the simulation's output or a fitted C_dA value.

- **Point-mass dynamics; quasi-static lean, no traction check.** The rider and bicycle are treated as a single point mass moving along the track's arc length. Lean angle is an algebraic function of instantaneous speed and curvature (Section 3.1), not a dynamical state with its own inertia, and is computed unconditionally from speed and curvature with no friction-circle/traction-limit check on whether the tire could actually supply the implied lateral friction.

- **Idealized (as-if-banked) lean geometry.** The rolling-resistance normal-force increase of Eq. (4) treats leaning as equivalent to riding a surface banked at angle θ (no lateral tire friction needed to supply the centripetal force), rather than modeling the actual friction-based lean mechanics of a bicycle on a flat track (Section 2.2).

- **CdA fit approximates wheel speed with center-of-mass speed.** The C_dA fit's distance-from-speed integration and lean-rate chain rule (Section 5.1) use the recorded (center-of-mass) speed directly, rather than the wheel-path speed v_{wheel} of Eq. (11) that the forward simulation uses for the same purpose; since $v_{\text{wheel}} - v \approx v\,\kappa h \sin\theta$ to leading order, the difference is first-order (linear) in $\kappa h \sin\theta$, and small in practice for realistic lean angles, but is not corrected for.

- **Flat-track assumption.** Neither the forward simulation nor the C_dA fit includes a gravitational grade term in Eq. (2). This is appropriate for a velodrome or flat closed-circuit use case, but a limitation for road data with meaningful elevation change.

- **No wind-speed correction.** `CdAFitter` uses ground speed directly in the drag equation (Eq. (6)); wind speed is present in recorded `.fit` telemetry but is not subtracted from or added to the speed used in the fit.

- **Steady environmental and mechanical conditions.** Rolling resistance coefficient, air density, and drivetrain mechanical loss (Sections 2.1 to 2.3) are each taken as a single constant value across an entire simulation run or fitting window, not varied over time or estimated from the data.

- **Unsmoothed finite-difference acceleration in the CdA fit.** The acceleration of Section 5.1 is a raw central difference of the recorded speed trace, with no low-pass filtering beforehand; noisy speed samples propagate directly into a noisy C_dA estimate.

- **Unweighted least squares.** The regression of Eq. (28) weights every sample equally and includes no outlier-robust reweighting, so a small number of corrupted samples can shift the fit disproportionately.

- **Idealized track geometry.** The track geometry of Section 4 is a two-straight, two-semicircle oval with a box-filter-smoothed (not clothoid) transition; it approximates, but does not exactly reproduce, the geometry of a real track or velodrome.

- **Position reconstruction is visualization-only.** The Cartesian centerline computed in Section 4.4 is used only to draw the track shape in the application; it has no influence on the physics of Sections 2 and 3.

A Notation, Units, and Parameters

Table 1 lists the symbols used throughout this document, their meaning, their SI unit, their default or typical value where one exists, and the corresponding function, class, or entity attribute in the implementation. Derivations throughout this document use SI units exclusively, with one display-only exception for reported velocity (Section 2.6).

Symbol	Meaning	Unit	Default / typical	Code reference
v	speed along direction of travel (center of mass)	m/s	–	`IPCalculator, state`
s	distance traveled along track (wheel path)	m	–	`IPCalculator, state`
t	elapsed time	s	–	`IPCalculator`
m	rider + bicycle mass	kg	100.0 (test value)	`Rider.weight_kg`
g	gravitational acceleration	m/s^2	9.806 65	`IPCalculator.GRAVITY`
F_{rr}	rolling resistance force	N	–	`IPCalculator._banked_rolling_resistance`
F_{ad}	aerodynamic drag force	N	–	`IPCalculator._ode_rhs (f_ad)`
F_p	propulsive force before drivetrain losses	N	–	`IPCalculator.calc_pedal_force`
N	normal force between tire and track under lean	N	–	`(derivation only)`
P_{report}	reported power output (force-capped)	W	–	`IPCalculator.solve (self.power)`
a_{power}	straight-line acceleration from rolling resistance, drag, and propulsive force	m/s^2	–	`IPCalculator._ode_rhs (a_power)`
C_{rr}	rolling resistance coefficient	1	0.002 (test value)	`Environment.crr`
ρ	air density	kg/m^3	1.12 (test value)	`Environment.air_density`
C_dA	aerodynamic drag area	m^2	0.195 (test value)	`Rider.cda`
$P(t)$	commanded or recorded power	W	–	`power plan / .fit power field`
$F_{\max}$	maximum possible force a rider can produce at low speeds	N	200	`IPCalculator.max_force`
η	fractional drivetrain mechanical loss	1	0.02 (test value)	`Environment.mech_losses`
θ	lean angle	rad	–	`IPCalculator._lean_angle`
κ	track curvature	1/m	–	`TrackShape.compute (kappa); IPCalculator._kappa_at`
h	rider center-of-mass height	m	optional	`Rider.com_height_m`
v_{wheel}	wheel-contact-path speed	m/s	–	`IPCalculator._wheel_speed`
r	corner radius	m	–	`TrackShape.compute (radius)`
z	center-of-mass height above a fixed reference	m	–	`(derivation only)`
a_{energy}	acceleration from the corner-lean energy exchange	m/s^2	–	`IPCalculator._energy_accel` computes only $a^{(s)}_{\text{energy}}$; the full a_{energy} is materialized in `CdAFitter._cda_for_s0`
C	implicit feedback coefficient (lean angle's dependence on speed)	1	–	`IPCalculator._energy_feedback_coeff`
L	total track length	m	250.0	`Environment.track_length`
c	fraction of track length devoted to cornering	1, $[0.20, 1.0]$	0.60	`Environment.corners`
ℓ_{straight}	arc length of each straight segment	m	derived from L, c	`TrackShape.compute (straight)`
ℓ_{corner}	arc length of each corner segment	m	derived from L, c	`TrackShape.compute (corner)`
–	curvature transition (smoothing) zone length	m	30.0	`TrackShape.transition`
ϕ	track heading angle	rad	–	`TrackShape.compute (heading)`
X, Y	Cartesian centerline coordinates (visualization only)	m	–	`TrackShape.compute (x, y)`
V_{eps}	low-speed floor for cornering/energy terms	m/s	0.1	`IPCalculator.V_EPS`
s_{race}	target race distance	m	4000	`IPCalculator.race_distance`
dt	output sampling interval	s	0.1 (sim) / 1 (class default)	`IPCalculator.dt`

Symbol	Meaning	Unit	Default / typical	Code reference
$t_{\max}$	solve time horizon	s	300	`IPCalculator.solve`
x, y	CdA-fit regression predictor (x) and response (y) variables	Pa; N	–	`CdAFitter._cda_for_s0`
s_0	lap-phase offset of the recorded window	m	–	`CdAFitter.fit (s0)`

Table 1: Notation, units, and default parameter values used throughout this document.